Übungsblatt 1: Speicherphysik & RAM-Typen

Mikrocomputertechnik 3 · Zellzustände, Ladung und Adressen auf Papier

Ziel und Arbeitsweise

Sechs Pflichtaufgaben sind vollständig auf Papier lösbar und üben Begründungen, Skizzen und Rechenwege wie in einer Klausur. Alle Zahlen und Adressbereiche sind ausdrücklich Lehrmodelle. Geben Sie Annahmen, Zwischenwerte und Einheiten an.

Bearbeitungszeit: etwa 75–90 Minuten selbstständig. **Orientierungspunkte:** 32, keine neue Prüfungsregel. Im Unterricht: Aufgaben 1–2 (10 min), Aufgabe 3 (15 min), Aufgabe 4 (15 min), Aufgaben 5–6 (15 min). Die Vorlesungsphasen enthalten Lösungsbesprechungen.

SRAM = Static Random-Access Memory, statischer wahlfreier Speicher. DRAM = Dynamic Random-Access Memory, dynamischer wahlfreier Speicher. 6T = sechs Zelltransistoren, 1T1C = ein Transistor und ein Kondensator. WL = Word Line, Wortleitung. BL / BLB = komplementäre Bit Lines, Bitleitungen. KiB = 2^{10} Byte, MiB = 2^{20} Byte, ein Byte enthält acht Bit. Die professionelle Speicherentscheidung unterscheidet Zelle, Zugriff und Software.

Aufgabe 1: Kapazität und gemeinsame Wege (5 Punkte)

1. Ein Speicher enthält 1 MiB Nutzdaten. Berechnen Sie Byte- und Bitzahl. Wie unterscheiden sich MiB und MB? (1 P)
2. Zählen Sie dafür die Zelltransistoren bei klassischem 6T-SRAM und die Transistoren und Kondensatoren bei 1T1C-DRAM. Warum folgt daraus noch keine genaue Chipfläche? (2 P)
3. Ein ausdrücklich angenommener gemeinsamer Port bedient entweder Befehlsholen oder einen Datenzugriff im betrachteten Takt. Beide werden gleichzeitig angefordert. Was ist der Konflikt? Beweist ein gemeinsamer Adressraum allein denselben Konflikt? (2 P)

Ihre Zählung und Erklärung:

Aufgabe 2: Flashänderung und Ausführungsweg (5 Punkte)

Logisches Flashmodell: gelöscht ist `FF`. Programmieren darf nur eins nach null ändern. Ein Löschen setzt alle Byte eines Modellblocks auf `FF`. Das ist keine Freigabe für beliebige Teilprogrammierung realer Bauteile.

1. Schreiben Sie `A5`, `81` und `AF` binär. Ist `A5` nach `81` ohne Löschen möglich? Und `A5` nach `AF`? (2 P)
2. Im selben Löschblock liegt ein benötigtes Byte `3C`. Beschreiben Sie ein korrektes Update des ersten Bytes von `A5` nach `AF`. (1 P)
3. Sechs **physische Löschzyklen** werden entweder als $6, 0, 0$ oder $2, 2, 2$ auf drei gleichartige Blöcke verteilt. Was verbessert sich, was wird dadurch nicht abgeschafft? (1 P)
4. Bedeutet ein NOR-Flash automatisch, dass daraus Code ausführbar ist? Nennen Sie die nötige Systembedingung für XIP, Execute-in-Place. (1 P)

Ihre Bitprüfung und Updatefolge:

Aufgabe 3: SRAM-Zustände und Reihenfolge (6 Punkte)

Die klassische 6T-Zelle besitzt zwei kreuzgekoppelte CMOS-Inverter und zwei nMOS-Zugriffstransistoren. CMOS = Complementary Metal-Oxide-Semiconductor. pMOS leitet im qualitativen Modell bei Gate null, nMOS bei Gate eins. Links steuert QB die Transistoren P_Q und N_Q, rechts Q die Transistoren P_QB und N_QB.

1. Füllen Sie die Tabelle für die zwei stabilen Zustände aus. Warum sind null/null und eins/eins keine stabilen digitalen Paare? (2 P)
2. Beschreiben Sie für Q null die Lesefolge mit Vorladung, WL und Leseverstärker. Welche Bitleitung sinkt? Muss Q vollständig unverändert in Volt bleiben? (2 P)
3. Schreiben Sie Q von null nach eins. Welche äußeren Bitleitungswerte werden benötigt? Ordnen Sie Datenbereitstellung, WL-Anstieg, Umschalten, WL-Abfall und Treiberfreigabe. (2 P)

Q	QB	P_Q	N_Q	P_QB	N_QB
0	1				
1	0				

Ihre Lese- und Schreibzeitfolge:

Aufgabe 4: DRAM-Ladung und Adressorganisation (6 Punkte)

Idealmodell: $C_{\text{cell}} = 30$ fF, $C_{\text{BL}} = 300$ fF, Plattenspannung null Volt und Bitleitung vorab 0,6 Volt. Die Vorsilbe f in fF bedeutet femto, 10^{-15}. Die Zelle beginnt wahlweise bei 1,2 oder 0 Volt. Ohne Verstärker gilt nach Verbinden:

$$V_{\text{share}} = \frac{C_{\text{cell}} V_{\text{cell}} + C_{\text{BL}} V_{\text{pre}}}{C_{\text{cell}} + C_{\text{BL}}}.$$

1. Berechnen Sie beide gemeinsamen Spannungen und die jeweilige Abweichung vom Vorladepegel. Warum sind Verstärken und Wiederherstellen nötig? (3 P)
2. Ordnen Sie Restore, WL-Absenken und Precharge. Wird beim Lesen der Zellkondensator immer vollständig entladen? (1 P)
3. Adressmodell: vier Banken, je 128 Zeilen, je 64 Spalten, je zwei Byte pro Spaltenelement. Bankbits $[15 : 14]$, Zeilenbits $[13 : 7]$, Spaltenbits $[6 : 1]$, Bytebit $[0]$. Bestimmen Sie Kapazität und Adresse für Bank zwei, Zeile fünf, Spalte neun, Byte eins. (2 P)

Ihre Ladungsrechnung und Bitfelder:

Aufgabe 5: Memory Map und Firmwareanfang (6 Punkte)

Byteadressiertes Lehrmodell: Flash ab `0x00000000`, 256 KiB; SRAM ab `0x10000000`, 64 KiB. Für einen `uint32_t`-Zugriff sind vier Byte und Ausrichtung durch vier nötig.

1. Geben Sie für beide Bereiche das exklusive Ende und das letzte Byte an. (1 P)
2. Prüfen Sie Vier-Byte-Zugriffe ab `0x1000FFFC`, `0x1000FFFF` und `0x10010000` getrennt auf Bereich und Ausrichtung. (1 P)
3. Firmwaremodell ohne Padding und weitere Abschnitte: `.text` 24 KiB, `.rodata` 4 KiB, `.data` 256 Byte, `.bss` 4 KiB und Stack 2 KiB. `.data` hat Initialwerte im Flash und Laufzeitobjekte im RAM; `.bss` enthält keine Nullbytes im Flash-Abbild. Berechnen Sie Bedarf und freien Platz beider Speicher. (2 P)
4. Wer kopiert `.data` und nullsetzt `.bss`? Erzwingt C-`const` allein die Platzierung im Flash? (2 P)

Ihre Grenzen, Speicherbilanz und Zuständigkeiten:

Aufgabe 6: FIFO und Speicherwahl (4 Punkte)

FIFO bedeutet First In First Out. Das Modell hat vier nutzbare Arrayplätze, Leseindex r, Schreibindex w und Belegungszahl count. Anfang: r = w = count = null. Bei Erfolg steigt die entsprechende Position modulo vier; bei voll oder leer bleibt ein abgewiesener Vorgang ohne Zustandsänderung. Alle Vorgänge erfolgen in einem Ausführungskontext.

1. Füllen Sie die Zustandsfolge aus. Geben Sie am Ende die Entnahmereihenfolge an. (2 P)
2. Ein weiteres Einlegen von F wird versucht. Ergebnis und Zustand? Warum beweist r gleich w allein keine Leere? (1 P)
3. Firmware soll Abschalten überstehen, häufig veränderte Zwischensummen nicht. Nennen Sie passende Speicherprinzipien und erklären Sie, warum „Code“ und „Daten“ keine Zellbauarten sind. (1 P)

Vorgang	ausgegeben	r	w	count
Anfang	–	0	0	0
A einlegen	–			
B einlegen	–			
C einlegen	–			
entnehmen				
D einlegen	–			
E einlegen	–			

Ihre Erklärung und Speicherwahl:

Optional: logische Modelle mit Host-GCC prüfen

Vom Repository-Hauptordner aus:

```
gcc -std=c11 -O2 -Wall -Wextra -Werror -pedantic \
    labs/src/lab01_memory.c -o /tmp/lab01_memory
/tmp/lab01_memory
```

Erwartete Ausgabe:

```
Flashmodell: A5 -> 81 erlaubt, A5 -> AF braucht Loeschen
Adressmodell: letzter ausgerichteter Wortstart 1000FFFC
Startupmodell: Initialwerte kopiert, BSS auf null
FIFO-Modell: volle und leere Zustaende korrekt
Alle Prueffaelle bestanden.
```

Untersuchen Sie die Fehlerfälle: Ein abgewiesenes Programmieren oder Einlegen ändert den Speicherzustand nicht. Das Programm dereferenziert keine Geräteadressen und modelliert keine analogen Zellströme oder reale Flash-Controller. Es arbeitet ohne Nebenläufigkeit.

Die typische Abschnittszuordnung der C-Folie können Sie unabhängig ansehen:

```
gcc -std=c11 -O0 -fno-pie -c labs/src/lab01_sections.c \
    -o /tmp/lab01_sections.o
readelf -SW /tmp/lab01_sections.o
readelf -sW /tmp/lab01_sections.o
```

Suchen Sie `.text`, `.rodata`, `.data` und `.bss` sowie die vier Symbole. Dies ist ein natives Host-Objekt, keine RISC-V-Firmware und kein Bootversuch. Die konkrete Toolchain kann die Abschnitte anders organisieren.